Week 11 & 12

✅ Process Synchronization

- Bounded-Buffer problem

✅ Race Condition

✅ Process Synchronization Software

- Solution to critical-section problem
- Peterson's Solution (Algorithm)
- Bakery Algorithm
- Semaphore
  - 2 types of semophore
  - semophore implementation (with no busy waiting)

✅ Classical Problems of Synchronization

- Bounded Buffer
- Reader-Writer problem
- Dining-Philosophers Problem

✅ Problems with Semaphores

✅ Semaphores programming

> ## Content

✅ Process Synchronization

- As we learn share memory, we see that when process need to exchange data, communicate by exchanging data, we need to make sure that the process can access data in right order → idea of process synchronization.

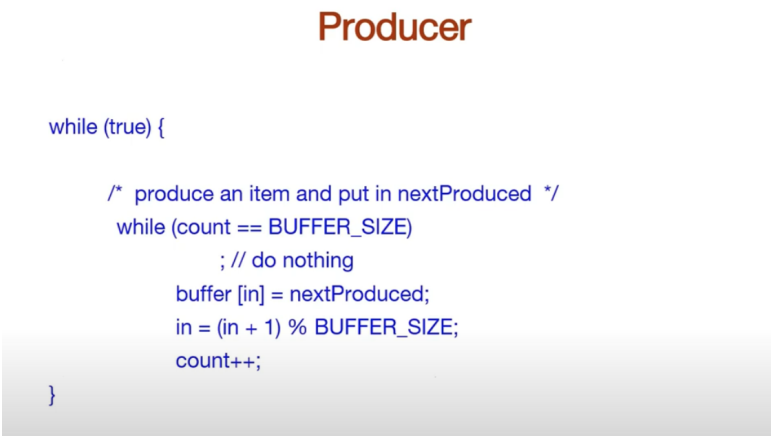
- for share memory, we need to think about this
- now, not only os, but also application with multi user, to make sure that the update of share data is done correctly

- if we don't have good process synchronization → data inconsistency

📍Bounded-Buffer Problem

When we talk about process synchronization → the example that we always use is producer-consumer problem (or bounded-buffer problem)

Manual way



- the idea is programmer said we gonna create another share varaible in addition to buffer → varaible name count
- count use to count the availble slot of the buffer.
- in point to slot that avaible at the moment
- every time producer write to buffer → it increment by one

- waiting in while loop

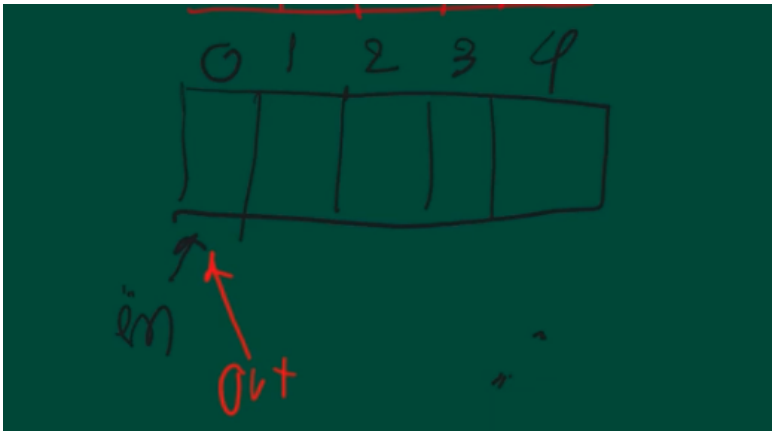
Consumer

```
while (true) {
        while (count == 0)
                ; // do nothing
                nextConsumed =  buffer[out];
                 out = (out + 1) % BUFFER_SIZE;
                 count--;

                    /*  consume the item in nextConsumed
}
```

- if can read data when count is not zero
  - count is not 0, → still new data
- out → position tha tconsumer can read the data.

count use as synchronization

> 👉 assuming that producer has chance to run first



producer check count is buffer size or not → in this case, count is 0 (not buffer size) → producer run → write data to first slot, increament in by 1.

- Assuming that OS allow producer to continue running → write second slot → after finish point to third slot
- Assuming OS let consumer run → read first slot → decrement count.....

(continue)

- come back and point at first slot dai.

If OS want producer to run, can't → wait until time out? prevent loss data.

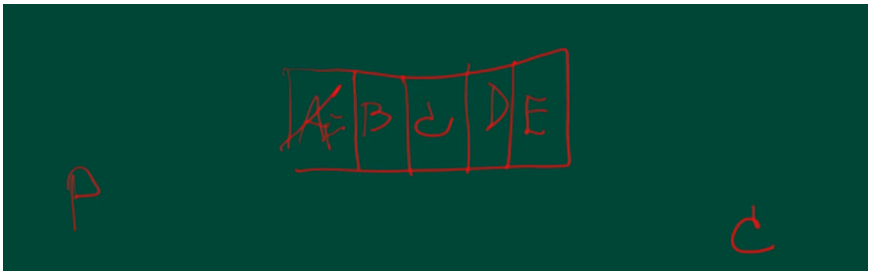
- Why we call it bounded buffer problem?
  - in bounded buffer problem, the situation is like this.
    - we have share data area → (buffer)
      - bounded → slot in each buffer is limited

  idea is we have 2 processes → producer and consumer
  - producer run forever to write data to this buffer, after write last slot, write first slot again
  - and customer start reading, read from first slot to last slot, then read first slot again

  if we allow this to work without synchronization, 2 thing can occur

  1. Lost data

  - what if producer sllow to run A B C D E and write F
    - and OS said producer stop, consumer start
    - consumer won't get A



  2. Duplicate data

  - producer write A B C D E, stop
    - consumer read A B C D E and OS allow consumer to continue read
    - get duplicate data

  That why we need to do synchronization.

> 👉 2 slides above is programmer way.

The programmer way seem to okay, but

(count is use as synchronization)

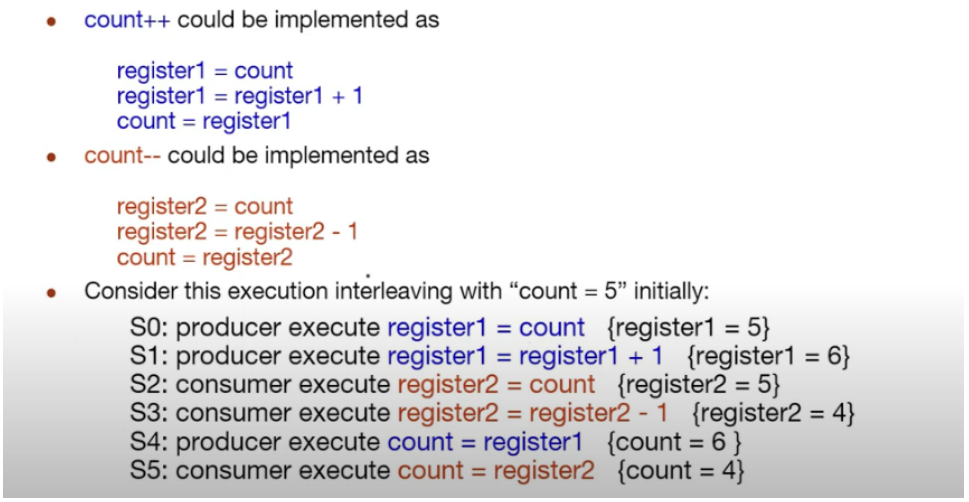
- If the producer has chance to run, it must finish count++
- if the consumer has chance to run, it must finish count—
- OS doesn't have chance to interrupt before count++, count—
  - but can we garantee this → NO, OS can interrupt anytime it want
    - cant garuntee that count++ will be finish first
- this why this algo not work

✅ Race condition

(the situation is that 2 or more process update share data at the same time → the result may not be correct because it based on the last process that finish)

- load
- increment
- store

How race condition occur?



read in here too. count should be 5

- if OS doesn't interrupt at some point, count can be 5.

✅ Process Synchronization Software

(idea to prevent race condition)

Lock count variable, even OS interrupt process, consumer still can't get value of count

- For a successful process synchornization:
  - Used resource must be locked from other processes until released
  - A waiting process is allowed to use the resource only when is released.
- A mistake could leave a deadlock

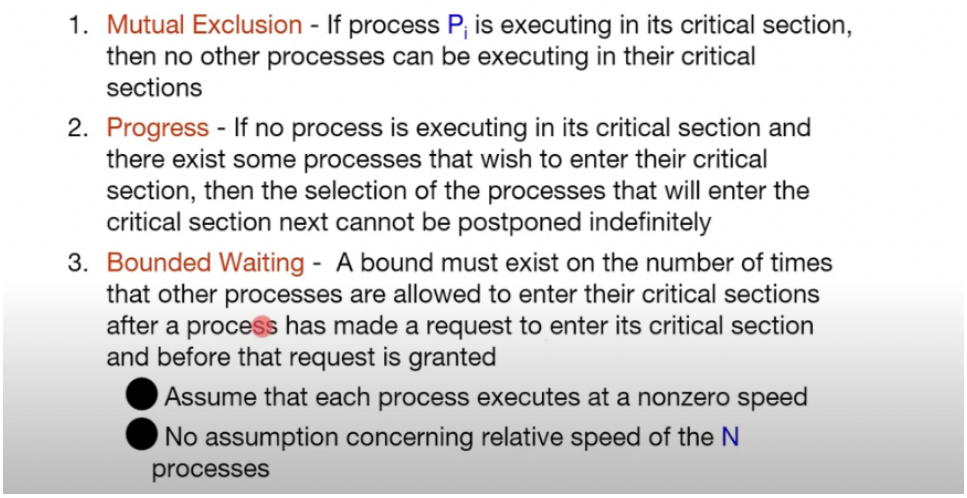
Some terms:

Critical region(section): A part of a program that updatre share data

A part of a program that must complete execution before other processes can have access to the resources being used.

- like count++ is critical region of producer
- count— is critical region of consumer

📍Solution to Critical-Section Problem



- develop your own algorithm to solve the problem, if you want it to be complete the algorithm must pass this 3.

👉 1. Mutual Exculstion - only 1 process can be in its critical region (require)

if we have 1 toilet, and it lock → pass, but have to consider if 2 people arrived at the toilet at the same time. → have to manage this condition (pass progress and bounded waiting

2. Progress - you should choose from the process that need to use critical section only.
3. Bounded Waiting - the process shouldn't wait forever, or like wait without hope.

👉 if you develop your own algorithm, the first one must pass → but maybe 2 nad 3 don't have to, it just not complete.

If algorithm can't pass number 2 → it can't pass number 3 too.

📍Peterson's Solution

(pass all 3, limitation → can work only for 2 process, can't work more than 2 processes

- Two process solution
- Assume that the LOAD and STORE instructions are atomic; that is, cannot be interrupted.
- The two processes share two variables:
  - int turn;
  - Boolean flag[2]
- The variable turn indicates whose turn it is to enter the critical section.
- The flag array is used to indicate if a process is ready to enter the critical section. flag[i] = true implies that process P_i is ready!

Algorithm for process Pi

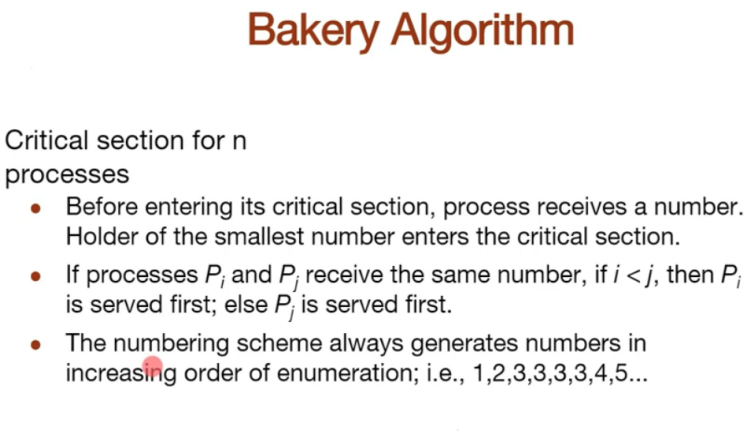
```
while (true) {
        flag[i] = TRUE;
        turn = j;
        while ( flag[j] && turn == j);

            CRITICAL SECTION

        flag[i] = FALSE;

            REMAINDER SECTION

}
```

if there more than 2 processes, peterson's don't know how to handle

📍Bakery Algorithm

(queue machine, when you go to bank or go to pay for telephone fee)

- like you go to queue machine, press button to get queue ticket)
- like the one who get the minimum number go first



- work the same way as queeuing machine → who want to sue service → get ticket → smallest number go first
- but the thing that this is different from queue machine is that bakery algorithm can't garuntee that every process have a unique number (processes arrived at the same time, get the same number)

How to solve problem?

- if 2 or more processes have the same ticket number, algo look at process id → let the one who ahve the smaller process id go first.

```
do {
    choosing[i] = true;
    number[i] = max(number[0], number[1], …, number [n – 1])+1;
    choosing[i] = false;
    for (j = 0; j < n; j++) {
                    while (choosing[j]) ;
                    while ((number[j] != 0) && ((number[j],j) < (number[i],i)) ;
    }
        critical section
    number[i] = 0;
        remainder section
} while (1);
```

1. pass the first one? only one choosen to use critical region
2. pass the second one? process that get pick is from process that got a queue
3. pass the third one? if you get smallest queue number or you are the lowest process id → noone can pass you → it your turn to run.

Bakery is a complete solution algorithm, Peterson too but got some limitation

The point is based on this algorithm, you gonna see this while loop

- the process of peterson's wait at `while(flag[j] && turn == j)` if the condition is not true.
- for bakery we have while loop to test the condition
    - when OS choose the process to run, if process can't pass condition it can still be in loop choosing[j] wait until time out. (waste system time)→ like we choose process that are not ready to run to wait? → this is called busy waiting
    - busy waiting → OS choose process that are not ready to run to running process → not OS fault → it doesn't wait for i/o or anything, like it still in loop and can't pass (os don't know this)
    - to solve problem, the process choose wait in waiting state

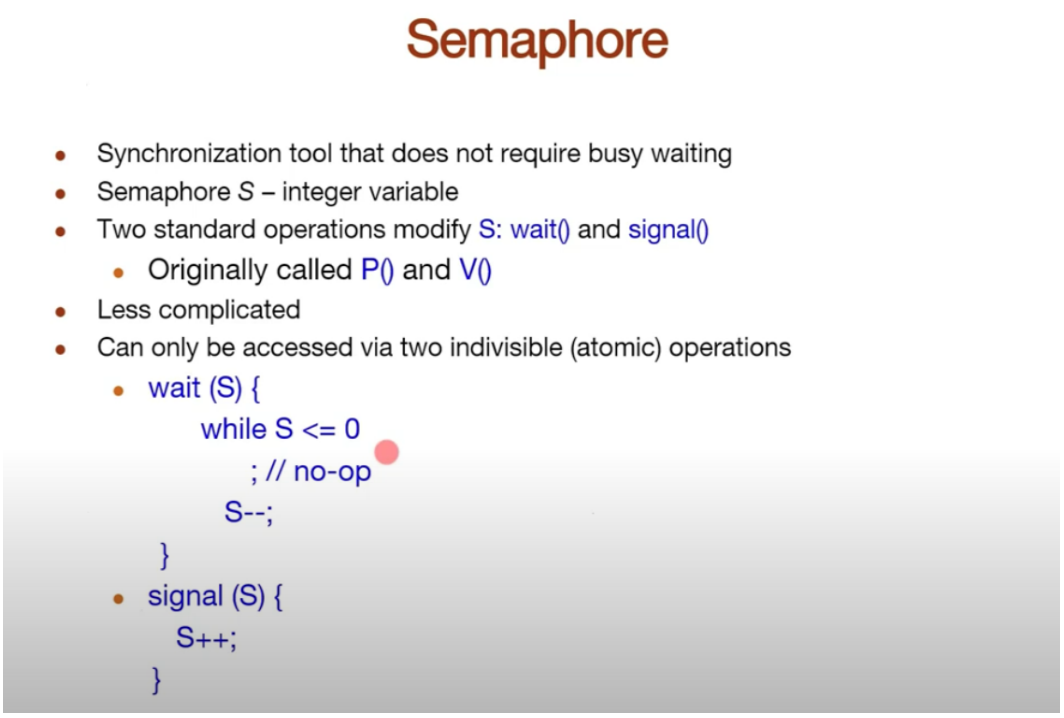
Bakery and Peterson require busy waiting → like we use loop to control the critical section (the process wait in loop)

📍Semaphore

(OS provide low level mechinism call semaphore to do synchronization iwthout the problem of busy-waitng)

Synchronization tool provide by OS, this semaphore doesn't have busy waiting

- because the process that wait for semaphore will be in the waiting state, and OS doesn't need to choose this process until it ready to go back to ready first.



- like the thing that i wait for is ready, i can go back to ready state.
- but the implementation is still busy waiting version → for explanation

assume when wait work → there no interrrupt (execute until finish)

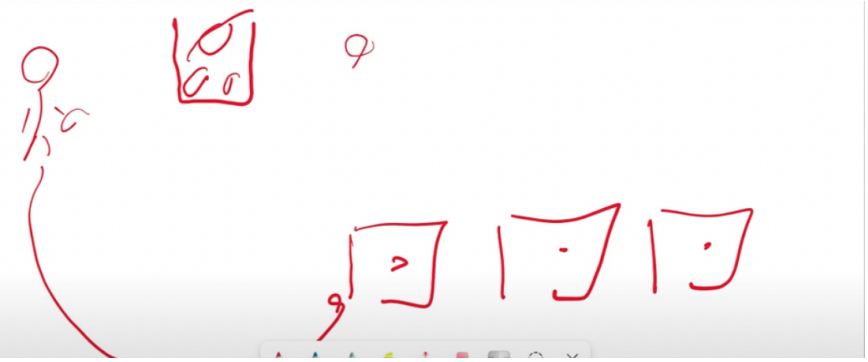
- wait, if no resources available, wait in loop, wait for wait to be postive

assume this for signal too

- signal increase the value of S

Like idea for

- wait is



- look at marble, take marble with you to toilet
- if there marble → some resources available
- like change number of marble based on your resources condition.

2 Types of Semaphore

- Counting semaphore – integer value can range over an unrestricted domain
- Binary semaphore – integer value can range only between 0 and 1; can be simpler to implement
  - Also known as mutex locks
- Can implement a counting semaphore S as a binary semaphore
- Provides mutual exclusion
  - Semaphore S; // initialized to 1
  - wait (S);

    Critical Section

    signal (S);

1. Counting - value of S can be any number
2. Binary - the value of S can only be 0 or 1 (only 1 marble)
   a. mutex lock → mutual exclusion → only 1 process at a time.
   b. to solve critical section problem → you need to use binary semaphore where you initialize semophore to 1, other process wait until the marble is returned.

Semaphore implementation (with no busy waiting)

Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue. Each entry in a waiting queue has two data items:
  - value (of type integer)
  - pointer to next record in the list
- Two operations:
  - block – place the process invoking the operation on the appropriate waiting queue.
  - wakeup – remove one of processes in the waiting queue and place it in the ready queue.

- os has special operation
  - block - when process called block, it get put in to the wating queue
  - wakeup - when process called wake up, os choose one from waiting queue to ready queue

Semaphore Implementation with no Busy waiting (Cont.)

- Implementation of wait:

```
wait (S){
    value--;
    if (value < 0) {
        add this process to waiting queue
        block(); }
}
```

- Implementation of signal:

```
Signal (S){
    value++;
    if (value <= 0) {
        remove a process P from the waiting queue
        wakeup(P); }
}
```

- wait, the one who want to enter check first → if the value is less than 0 → no space available → go to block (wait for process that in critical section finish and call signal.

Semophore

- pass mutual exclusion → 1 process at a time
- garuntee progress - the one who want to enter call wait, os only choose process in waiting queue(process tthat call block)
- but is it pass bounded waiting → depend
  - if OS implementation of signal → first come first serve → pass
  - but if OS get priority → OS that have no priority might have to wait forever.

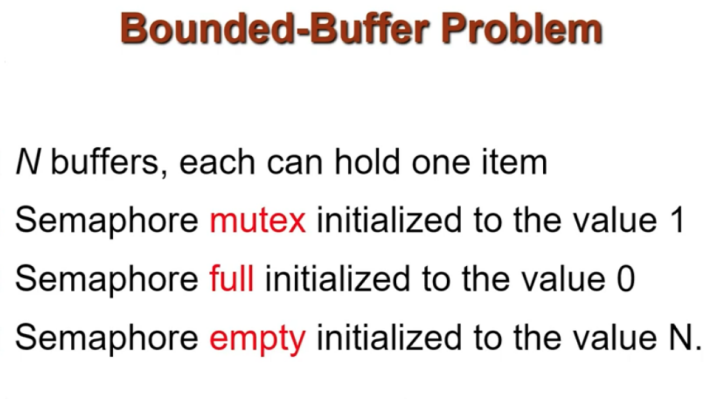
✅Classical Problems of Synchronization

(how to use samophore to solve this problem)

the way to use semaphore depneds on programmer algorithm

1. Bounded-Buffer Problem
2. Readers and Writers Problem
3. Dining-Philosophers Problem

📍Bounded Buffer



- for the version that we try to solve our self → busy waiting, count var → not really work.

for this algorithm

- we have share data that is a array of n slot
  - we share this semahore between 2 processes

- mutex is a binary semaphore → and initialize to 1 → put 1 marble in jar
- full is counting semaphore → start 0, max value is number of slot
- emty is also counting semaphore → start with n marble in a jar

Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
while (true) {

        // produce an item
    wait (empty);
    wait (mutex);

        // add the item to the buffer

    signal (mutex);
    signal (full);
}
```

`wait(empty)` - slot available (while there some marble in a jar)

`wait(mutex)` - to garuntee that when producer work, consumer can't work

- producer finish writing first before allow consumer to read the data

after it finish entering the data.

`signal(mutex)` - return back the marble to mutex jar

`signal(full)` - it take 1 marble from empty jar but it return that marbe in to a full jar.

- because for consumer, it check for full jar first → if there no marble, it mean no data to read (tell consumer that new data is coming)
    - and then like check if producer write something → if no → go read → and return marble to empty jar

empty and full jar → kindof now use to send signal

📍Reader-Writers Problem

Readers-Writers Problem

- A data set is shared among a number of concurrent processes
  - Readers – only read the data set; they do not perform any updates
  - Writers – can both read and write.
- Problem – allow multiple readers to read at the same time. Only one single writer can access the shared data at the same time.
- Shared Data
  - Data set
  - Semaphore mutex initialized to 1.
  - Semaphore wrt initialized to 1.
  - Integer readcount initialized to 0.

- can only use 2 semaphore
  - mutex - binary semaphore
  - wrt - binary semaphore

- The structure of a writer process

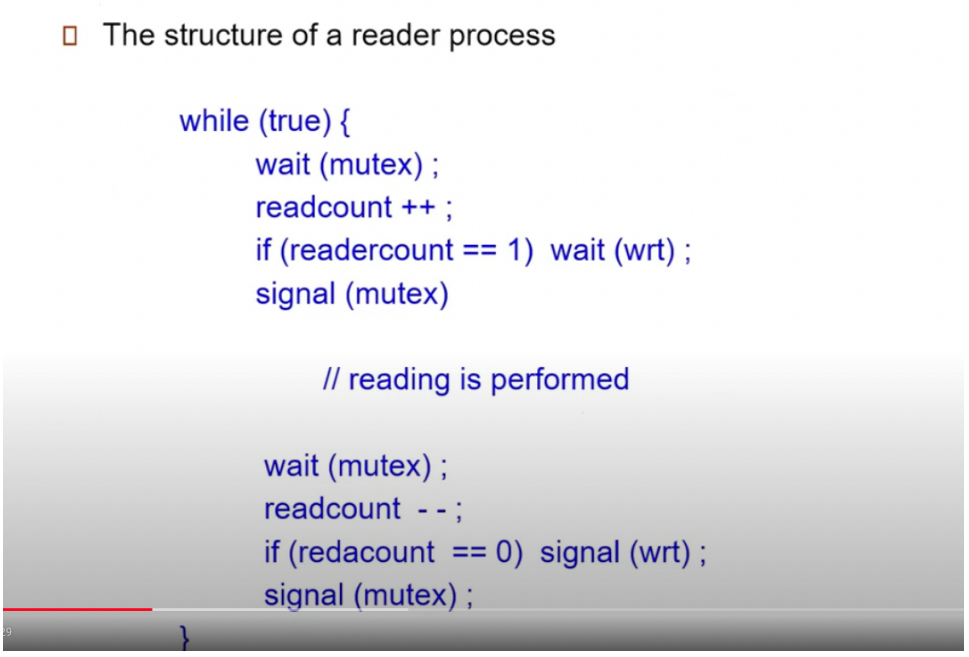
```
while (true) {
        wait (wrt) ;

            //    writing is performed

        signal (wrt) ;
}
```

- `wrt` is to prevent between reader and writer
  - when reader is reading, writer can't read

- when writer is writing, reader can't read

writer is binary semaphore as well, use between writer and reader

like it has to wait

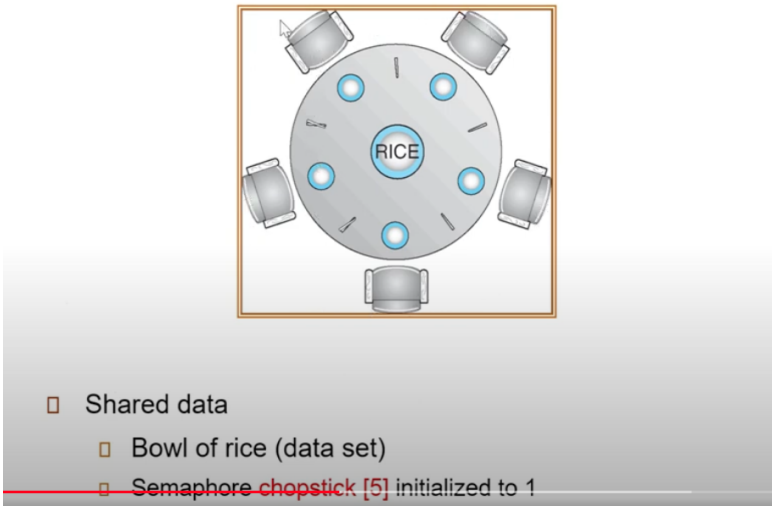


- `mutex` is use to increment the number of reader, and decrement the number of writer.
- in this program, we allow more than 1 reader
  - mutex garuntee that readcount++, can finish the increment process (like finish without problem)
- like i'm the first one, increment readcount to 1, and then check if writer still writing `wait(wrt)`, then put to waiting state
- if finish → `signal (mutex)`

17.49 29(clip 2)

📍Dining-Philosophers Problem

(when a process need more than 1 resources to finish)



- if hungry → eat, if non → think about philosophy
- the thing is to be able to eat → have chopstick
  - need 2 resources for process to finish each of it tasl → lefyt and right chopstick
  - for this example, we want to use semaphore to prevent 1 philosopher to whole the same chopsticks
  - when a chopsticks is in 1 philosopher hand → next one has to wait

Way to solve

The structure of Philosopher *i*:

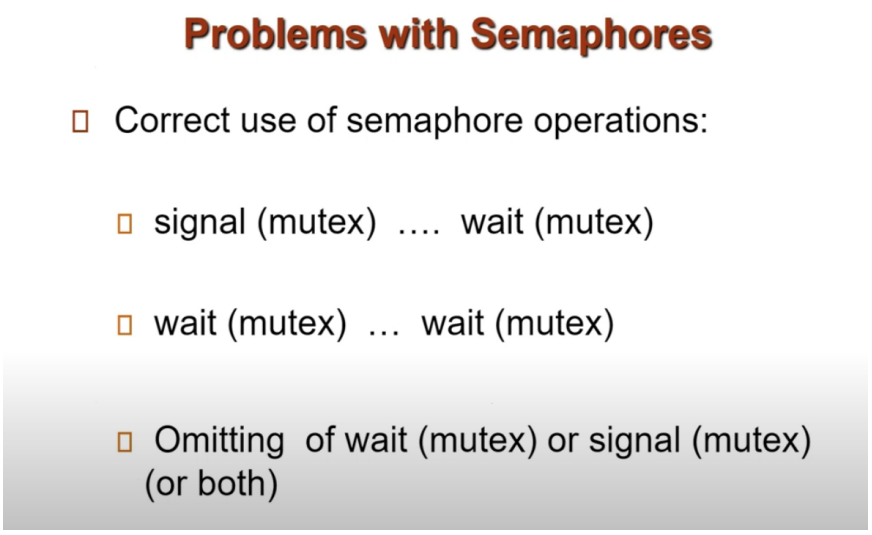
```
While (true) {
        wait ( chopstick[i] );
        wait ( chopStick[ (i + 1) % 5] );

            // eat

        signal ( chopstick[i] );
        signal (chopstick[ (i + 1) % 5] );

            // think

}
```

- check chopstick from the left hand → if there are avaible, pick up chopstick to his left hand
- check right → if avaiable, pick up and eat, if no → wait
- `signal(xhopstick[i])` → if finish, out down left chopstick
- `signal(chopstick[(i+1) % 5])` put down right chopstick

if they wait for each other to put down → deadlock.

- the point is synchronization is needed, but if do it without care might cause deadlock

✅ Problems with Semaphores



- signal(mutex)....wait(mutex)
  - if you call signal first → you put another marble in a jar that suppose to have 1 marble
- wait(mutex)...wait(mutex)
  - no compile relation error to tell you about this
- .....

> The solution to this problem is maybe monitor

✅ Semaphores programming

- pthread library is easier to understand than looking at semaphore directly

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>

int count = 5;
sem_t mutex;
void *increment(void *param);
void *decrement(void *param);
int main() {
    pthread_t tid1, tid2;
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    sem_init(&mutex, 0,1);
    pthread_create(&tid1, &attr, increment, NULL);
    pthread_create(&tid2, &attr, decrement, NULL);
    /* now wait for the thread to exit */
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    printf("count  = %d\n", count);
    sem_destroy(&mutex);
}

void *increment (void *param) {
    int temp;
    sem_wait(&mutex);
    temp = count;
    sleep(rand() % 5);
    temp++;
    count = temp;
    sem_post(&mutex);
    pthread_exit(0);
}

void *decrement (void *param) {
    int temp;
    sem_wait(&mutex);
```

```
    temp = count;
    sleep(rand() % 5);
    temp--;
    count = temp;
    sem_post(&mutex);
    pthread_exit(0);
}
```

- create 2 thread
  - 1 thread call increment, other call decrement
- then parent wait for its children

count and mutex is share variable

- mutex is semaphore

`sem_init(&mutex, 0, 1)`

- 0 ⇒ this semophore is share by only the thread created by this process, not share between
- 1 ⇒ mutex start by 1

`sem_wait` → wait

`sem_post` → signal

instead of using count++,

mimic load⇒ `temp=count`

incrementation ⇒ `temp++`

store ⇒ `count=temp`

Output(using semophore)

```
sarun@sarun-flowx13:~/os_course/pthreadsemaphore$ gcc -o semthread semthread.c -l pthread
sarun@sarun-flowx13:~/os_course/pthreadsemaphore$ ./semthread
count  = 5
sarun@sarun-flowx13:~/os_course/pthreadsemaphore$ ./semthread
```

Output(not using semophore)

- comment out sem_wait....

```
sarun@sarun-flowx13:~/os_course/pthreadsemaphore$ gcc -o semthread semthread.c -l pthread
sarun@sarun-flowx13:~/os_course/pthreadsemaphore$ ./semthread
count  = 6
sarun@sarun-flowx13:~/os_course/pthreadsemaphore$
```

- it maybe 6 or 4 (not correct)

Quiz

1. The situation that is more than one process access and update shared data at the same time without proper synchronization is called ____.

- Race condition

2. If the shared variable name banana store the initial value of 7, process A was executed once to increase the value of the banana by 2. Process B was executed once to decrease the value of the banana by 1. If a proper synchronization was used, the final value of banana is _____.

- 8 (if everything work fine)

3. If the shared variable name banana store the initial value of 7, process A was executed once to increase the value of the banana by 2. Process B was executed once to decrease the value of the banana by 1. If a proper synchronization was not used, the final value of banana is _____.

- A or C or D (6, 8, or 9)
    - because if 6 happen, there something wrong, the one who finish last is the one who decrease....
    - why 8 still can occur → the value can be 8

4. The main idea of proces synchronization to manage the shared resource is to lock the resource that it si using from other processes.
    - True
5. A part of a program that must complete execution before other processes can have access to the resources being used.

- C or D (Critical region, Critical section)

6. For the algorithm to solve the problem in the previous question, the condition that must ensure that if a process is executing in it scritical section, then no other process can be executing in their critical section is called _____.
    - Mutual Exclusion
7. For the algorithm to solve the problem in question 5 , the condition that must ensure that the processes that will be selected to use their critical sections must be the processes that are willing to use their critical section is called_____.
    - Progress
8. For the algorithm to solve the problem in question 5, the condition that must ensure that the processes waiting to enter their critcial section will not wait indefinitely is called_____.
    - Bounded waiting
9. The name of the algorithm proposed to solve the critical section but has the limitation that it can support synchronization between 2 processes is _____.
    - Peterson’s Algorithm
10. The name of the algorithm proposed to solve the critical section problem that can support synchronization among any number of process and use the idea of queue number machine si ___.
    - Bakery Algorithm
11. When a process is working in its critical section, OS cannt interrupt the process.
    - False (it can, but the resource is lock,..)
12. For Peterson’s algorithm the variable “turn” is sued for progress and the variable flag is used for bounded waiting?
    - not yet
13. Bekery algorithm allows more than one process to have the same queue numbre
    - True (if this happen → the least id?)

14. Require a process to get a queue number in Bakery algorithm can ensure bounded waiting.
    - False
15. The order of queue number in Bakery algorithm support s bounded waiting.